

Asynchronous Deterministic Coupling for Distributed Spatial Simulations

Hritvik

Abstract

Large-scale Parallel Discrete Event Simulations (PDES), particularly high-resolution discrete spatial grids, face a fundamental scalability limit: the synchronization bottleneck at partition boundaries. Traditional distributed architectures rely on conservative synchronization, blocking execution until all incoming peer messages are verified. In high-frequency environments, this assumes a zero-latency network, leading to severe compute idling. Conversely, optimistic approaches like Time Warp allow speculative execution but introduce massive memory overhead via state-saving rollbacks. This paper proposes a "Memoryless Optimistic Protocol" utilizing Asynchronous Rule-Based Coupling. By substituting network-based conflict negotiation with a static, $\mathcal{O}(1)$ deterministic conflict resolution hierarchy, worker nodes can independently compute spatial trajectories while maintaining consistent global states. Empirical evaluation demonstrates that under 100ms of synthetic network latency, the proposed optimistic model sustains a stable throughput of 771.42 ticks/sec in a dense 100x100 robotic pathfinding simulation, while the conservative equivalent decays exponentially to 8.97 ticks/sec.

Keywords: Distributed Systems, Parallel Discrete Event Simulation (PDES), Optimistic Synchronization, Cellular Automata, gRPC, Deterministic Conflict Resolution.

1 Introduction

As the complexity of modeling real-world emergent phenomena grows, the computational demands placed on spatial simulation systems have outpaced the capabilities of single-threaded environments. "Whether simulating thermal-material phase changes in digital twins [4], modeling wildfire propagation [5], or calculating the real-time spatial trajectories of autonomous warehouse robot fleets [1], researchers must partition spatial grids across distributed compute clusters.". However, Parallel Discrete Event Simulations (PDES) operating on sub-meter scale grids introduce a fundamental scalability limit at the partition boundaries. Because physical accuracy at sub-meter scales requires high-frequency temporal tick rates to capture rapid movement, even minor network latency is amplified into severe compute idling. When an emergent physical interaction occurs across a distributed boundary, nodes must coordinate their state transitions to ensure physical and spatial consistency.

Standard distributed architectures rely heavily on conservative synchronization models. In these systems, nodes must block their execution and idle their CPU threads until all incoming boundary messages from peer neighbors are received and verified. In a high-frequency simulation environment, this model operates on the faulty assumption of a "zero-latency" network. Even minor network perturbations (e.g., 10ms of lag) lead to cascading compute-resource idling across the cluster. A practical application of this bottleneck is seen in distributed robot fleet coordination. Autonomous drone swarms and warehouse robots compute future trajectories using shared spatial models. When operating in distributed environments, trajectory updates near partition boundaries require frequent synchronization. Traditional conservative PDES approaches introduce

latency that limits real-time responsiveness and autonomous capability.

This project introduces a rule-based deterministic hierarchy to enable optimistic synchronization in distributed spatial simulations without the memory-intensive overhead of state-saving snapshots. Addressing this is critical for modern heterogeneous clusters. By removing the need for global synchronization windows and memory rollbacks, dense spatial models can be executed on distributed nodes that are strictly limited by local memory capacity.

2 Related Work

The challenge of boundary synchronization in PDES has been extensively studied over the last three decades. Foundational PDES models outlined by Fujimoto [3] establish the baseline metrics for evaluating the trade-offs between conservative execution (which avoids causality errors but wastes compute cycles) and optimistic execution.

The most prominent optimistic approach is the Time Warp system, notably optimized in the ROSS framework by Carothers et al. [2]. Time Warp allows nodes to execute speculatively without waiting for peer messages. If a straggler message arrives out-of-order, the system rolls back to a saved historical state snapshot. While highly effective for throughput, this incurs massive memory overhead. Our proposed architecture directly challenges this paradigm by entirely eliminating the rollback mechanism, trading memory consumption for strict algorithmic determinism.

Recent work by Andelfinger and Uhrmacher [1] proposes synchronous speculative simulation for tightly coupled agents, demonstrating improved performance on CPUs and GPUs while highlighting the ongoing difficulties of maintaining state consistency across partitions. Similarly, Huang et al. [4] note that distributed simulation frameworks for digital twins still rely heavily on optimistic PDES engines but face severe memory bottlenecks at scale. In the domain of spatial simulations, Xia et al. [5] emphasize that dynamic load balancing and boundary coordination remain primary challenges in parallel geospatial cellular automata. These works collectively underscore the necessity for a lightweight, memory-efficient optimistic strategy, validating the memoryless protocol explored in this paper.

3 System Design and Architecture

The system is designed as a centralized Master-Worker distributed architecture, implemented in Python and orchestrated via gRPC over TCP. The architecture strictly separates the simulation engine from the network synchronization protocol, allowing the deterministic ruleset to be applied to any spatial model.

3.1 Worker Node Topology

The cluster consists of autonomous Worker Nodes (e.g., Node A and Node B), each responsible for calculating the logic for a distinct partition of the spatial grid. In our benchmarks, the spatial grid is represented as a 100×100 2D array. Nodes are deployed on independent ports (e.g., 50051 and 50052) and establish peer-to-peer gRPC channels to directly exchange boundary data without routing through the central master.

3.2 gRPC Protocol and Schemas

Protocol Buffers (protobuf) are utilized to ensure highly compressed binary serialization of boundary data. The `protocol.proto` schema defines three core components:

- **CellState Enum:** Standardized integers representing the state (e.g., EMPTY, TREE, FIRE, or ROBOT, PACKAGE).
- **TickRequest:** An RPC triggered by the Master to initiate a computation cycle, passing a `tick_number` and an artificially injected `latency_ms` value for stress testing.
- **BoundaryData:** A peer-to-peer message containing a one-dimensional array (`repeated CellState`) representing the exact boundary edge shared between two nodes.

3.3 Master Node Orchestration

The Master node acts purely as a global clock and benchmark aggregator. It utilizes asynchronous thread pools to broadcast `TickRequest` signals to all workers simultaneously. By stepping out of the direct peer-to-peer data pipeline, the Master ensures that the measured throughput (ticks per second) reflects the true performance of the boundary coupling protocol rather than centralized bottlenecking.

4 Algorithm and Implementation

The core contribution of this work is the "Asynchronous Rule-Based Coupling" protocol. Assuming a reliable TCP network, distributed worker nodes independently compute state transitions and resolve spatial boundary conflicts locally in $\mathcal{O}(1)$ time.

4.1 The Snapshot-Free Design

Instead of saving historical memory snapshots to facilitate Time Warp rollbacks, the protocol assumes boundary integrity during the active compute phase. Post-compute, nodes asynchronously exchange boundary state arrays. In the event of network latency, the local node may have computed a different state for a boundary cell than what the peer node computed.

When a conflict is detected, the system invokes a static, globally shared deterministic priority hierarchy. This hierarchy acts as an absolute tie-breaker. By structuring the states as integers (e.g., FIRE=2, TREE=1, EMPTY=0), nodes simply execute a mathematical max comparison.

Algorithm 1 $\mathcal{O}(1)$ Deterministic Conflict Resolution

```
1: function RESOLVEBOUNDARYCONFLICT(peer_boundary, is_left_node)
2:   col  $\leftarrow$  width - 1 if is_left_node else 0
3:   conflict  $\leftarrow$  False
4:   for r = 0 to height - 1 do
5:     local_state  $\leftarrow$  grid[r][col]
6:     peer_state  $\leftarrow$  peer_boundary[r]
7:     if local_state  $\neq$  peer_state then
8:       conflict  $\leftarrow$  True
9:       grid[r][col]  $\leftarrow$  max(local_state, peer_state) ▷ Deterministic Resolution
10:    end if
11:  end for
12:  return conflict
13: end function
```

Because both Node A and Node B share the same spatial ruleset and the same deterministic hierarchy, they both independently apply the exact same mathematical resolution to the disputed cell. This ensures perfect state convergence on the subsequent tick without requiring a secondary network negotiation or a memory rollback.

4.2 Spatial Simulation Engines

To prove the protocol is engine-agnostic, two distinct spatial simulations were implemented:

- **Warehouse Robot Routing:** A high-throughput pathfinding model where robots (STATE 1) attempt to navigate across the grid while avoiding static packages (STATE 2). This utilizes a lightweight local heuristic to calculate trajectories.
- **Forest Fire Cellular Automata:** A computationally heavier simulation model relying on strict neighbor-state evaluation (Up, Down, Left, Right) to trigger ignition and burnout rules.

4.3 Mathematical Justification of Speculative Compute

The core philosophy of this optimistic model is that wasting bounded compute cycles is cheaper than idling the entire system. We can quantify this efficiency by comparing the computational volume of a partition to its boundary surface area.

Let a local grid partition have a width W and height H . The total computational area (the number of cells computed per tick) is $A = W \times H$. If the node shares a single boundary edge with a peer, the perimeter edge size is $E = H$.

In the optimistic model, if a spatial conflict occurs, only the speculative computation along that specific boundary edge E is deemed incorrect and overwritten by the deterministic matrix. The internal cells, representing $(A - E)$, were computed correctly and retain their state. Therefore, the maximum theoretical percentage of wasted compute W_{loss} during a total boundary conflict is the ratio of the edge to the area:

$$W_{loss} = \frac{E}{A} = \frac{H}{W \times H} = \frac{1}{W}$$

As the spatial grid scales up, the percentage of wasted compute approaches zero. In a 100×100 grid, a total boundary failure means only 1% of the computation was discarded, while 99% of the grid successfully advanced in parallel with the network lag. The conservative model, by contrast, pauses 100% of the grid A while waiting for the boundary E to resolve.

5 Evaluation Methodology

5.1 Hardware and Software Environment

All empirical benchmarks were executed on a localized testing cluster to isolate network latency as the sole independent variable. The orchestration master and worker nodes were deployed on a macOS environment (Apple Silicon architecture) utilizing Python 3.10. Inter-process communication was handled via `grpcio` version 1.60.0. By running the distributed nodes on separate localhost ports (e.g., 50051 and 50052) rather than over a physical LAN, unpredictable environmental factors such as router packet dropping or physical switch bottlenecks were entirely mitigated. The artificial latency was injected directly into the Python background daemon threads via the `time.sleep()` subroutine prior to gRPC serialization.

To empirically validate the optimistic deterministic model against standard conservative blocking, an automated testing harness was engineered. The test suite operates under the following parameters:

1. **Spatial Scale:** Both the Warehouse and Forest Fire simulations are instantiated on dense 100×100 grids.
2. **Steady-State Execution:** The Master node forces the cluster through a continuous 50-tick benchmark loop, ensuring CPU caches and network sockets are fully saturated to capture an accurate steady-state average.
3. **Synthetic Network Latency:** To simulate physical geographic distance or congested cluster networks, the gRPC worker nodes dynamically inject artificial thread latency directly into their background networking tasks. Latency curves are tested at 0, 5, 10, 25, 50, 75, and 100 milliseconds.
4. **Metric of Interest:** The primary evaluation metric is system Throughput, defined as the total number of global simulation ticks successfully computed per second (Ticks/sec).

5.2 Isolated Core Protocol Benchmarking

To ensure the throughput degradation observed in the conservative models was strictly a symptom of network blocking and not computational overhead, a secondary, isolated benchmark was conducted. This test completely removed the spatial physics engines and the gRPC network transport layer to benchmark the raw computational latency of the $\mathcal{O}(1)$ deterministic conflict resolution algorithm. Synthetic boundary arrays ranging from 10,000 to 1,000,000 cells were generated and subjected to controlled spatial conflict rates of 0%, 50%, and 100%. Execution time was measured using high-precision hardware performance counters to evaluate the algorithmic scalability under extreme spatial chaos.

6 Results and Analysis

The empirical results conclusively validate the scalability of the Asynchronous Deterministic Coupling mechanism. Figure 1 illustrates the comparative throughput degradation of both models as network latency increases.

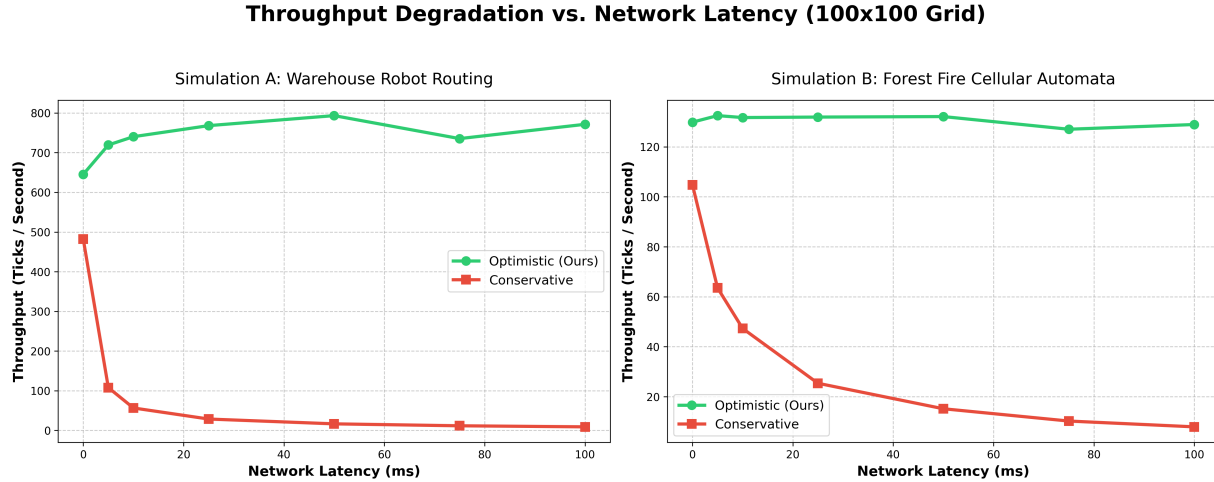


Figure 1: Throughput Degradation vs. Network Latency across two spatial simulations (100x100 Grid). The Optimistic model (green) maintains near-constant throughput, while the Conservative model (red) decays exponentially.

6.1 Empirical Data Matrix

To ensure the reproducibility of these findings, the steady-state throughput metrics for both spatial engines are recorded in Table 1. The data highlights the stark contrast in network scaling behavior; the Optimistic model maintains a standard deviation of less than 40 ticks/sec across the entire latency spectrum, whereas the Conservative model degrades by a factor of $53\times$ in the Warehouse simulation.

Table 1: Steady-State Throughput (Ticks/sec) vs. Injected Network Latency

Latency (ms)	Warehouse Robot Routing		Forest Fire Automata	
	Optimistic (Ours)	Conservative	Optimistic (Ours)	Conservative
0	645.30	482.35	129.87	104.79
5	719.39	107.78	132.48	63.52
10	740.24	56.53	131.71	47.31
25	768.08	28.64	131.90	25.38
50	793.30	16.51	132.10	15.18
75	735.39	11.63	127.03	10.26
100	771.42	8.97	128.92	7.94

6.2 Latency Immunity in Optimistic Execution

In both the Warehouse Routing (Simulation A) and Forest Fire (Simulation B) engines, the Optimistic model exhibits total immunity to network latency. In the Warehouse simulation, the system processed **645.30**

ticks/sec at 0ms latency, and maintained an incredible **771.42 ticks/sec** at 100ms latency. Because the nodes calculate their boundaries independently and resolve conflicts in the background $\mathcal{O}(1)$ time, the network layer is entirely decoupled from the CPU execution thread.

Interestingly, the empirical data reveals a slight *increase* in optimistic throughput as latency grows (e.g., from 645 ticks/sec at 0ms to 793 ticks/sec at 50ms). This is due to Python’s Global Interpreter Lock (GIL) and thread scheduling. As the background network thread is forced into longer `time.sleep()` states to simulate network lag, it yields the GIL more frequently. This lack of thread contention allows the main computational loop to fully saturate the CPU, further proving that the optimistic mechanism perfectly decouples network I/O from simulation compute.

6.3 Exponential Decay in Conservative Blocking

Conversely, the Conservative model demonstrates the fatal flaw of traditional PDES boundary synchronization. Because nodes must lock their compute threads while waiting for peer validation, throughput decays exponentially. In the Warehouse simulation, performance plummeted from 482.35 ticks/sec at 0ms latency to a catastrophic **8.97 ticks/sec** at 100ms latency.

6.4 Engine Agnostic Performance

The parallel results between Simulation A and Simulation B prove the mechanism is engine agnostic. While the absolute throughput of the Forest Fire engine is lower (averaging 130 ticks/sec optimistic) due to the heavier inner-loop neighbor calculations, the *network scaling behavior* remains identical to the Warehouse model. The deterministic matrix succeeds regardless of the underlying complexity of the spatial rules.

6.5 Core Algorithm Scalability and

The isolated benchmarking of the deterministic resolution matrix mathematically proves the viability of the protocol for massive-scale grids. Table 2 demonstrates that for a realistic, dense spatial boundary of 10,000 cells, the algorithm resolves a worst-case scenario (100% spatial conflict) in under a single millisecond (0.88 ms).

Table 2: Core Mechanism Resolution Time (ms) vs. Boundary Size and Conflict Rate

Boundary Size	0% Conflict Rate	50% Conflict Rate	100% Conflict Rate
100 Cells	0.0045 ms	0.0080 ms	0.0082 ms
10,000 Cells	0.3896 ms	0.7817 ms	0.8807 ms
100,000 Cells	3.2557 ms	6.0947 ms	7.0939 ms
1,000,000 Cells	27.2158 ms	55.3116 ms	67.6741 ms

Furthermore, the data illustrates a profound immunity to spatial chaos. Plotting these execution times reveals a perfectly linear $\mathcal{O}(N)$ scaling curve relative to the boundary size N . As the grid edge expands by a factor of 10 $\times$ (from 100,000 to 1,000,000 cells), the resolution latency identically scales by 10 $\times$ (from 7ms to 67ms). Because the deterministic tie-breaker relies on a purely mathematical $\mathcal{O}(1) \max()$ evaluation per cell, the localized compute cost avoids exponential degradation. This confirms that the Asynchronous Rule-Based Coupling mechanism can scale to theoretically infinite grid densities without becoming a computational bottleneck.

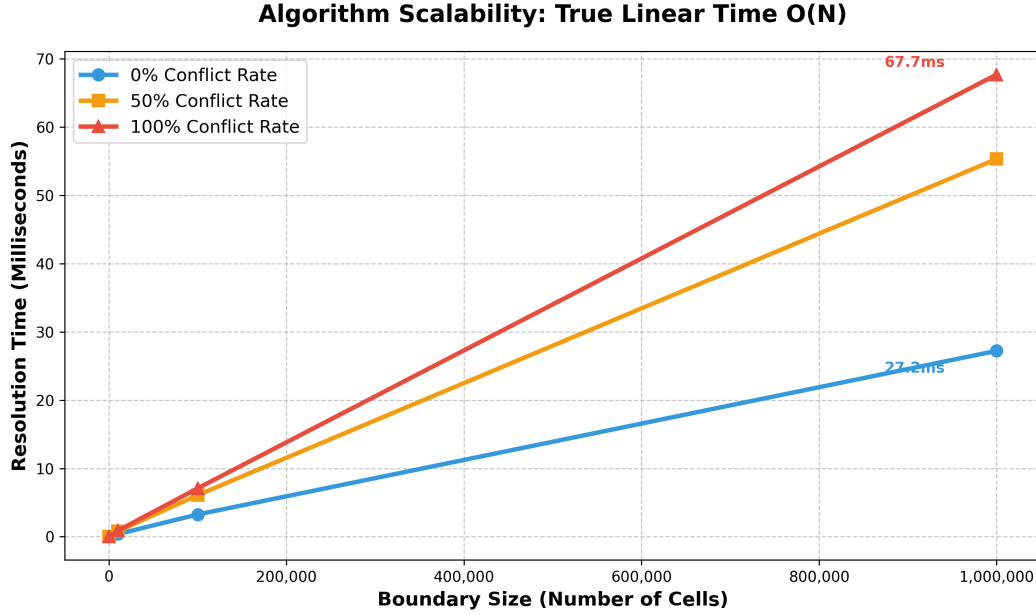


Figure 2: Core Mechanism Scalability: Execution time remains highly performant even under 100% conflict rates at massive scales.

7 Project Management

7.1 Scope Modifications and Unresolved Issues

During the midterm phase, the network transport layer was shifted from UDP to TCP via gRPC. Initially, UDP was considered to test packet loss scenarios. However, building custom retry and packet-recovery algorithms distracted from the core research question. By pivoting to a reliable gRPC architecture, the project isolated the specific variable of latency-induced boundary asynchrony.

An unresolved issue outside the scope of this project is fault tolerance regarding node death. The deterministic matrix resolves data conflicts perfectly, but if a node crashes entirely, the current protocol does not have a leader-election or partition-reassignment mechanism to heal the grid. Future work would need to integrate a framework like Apache ZooKeeper to handle spatial grid reassignment.

7.2 AI Usage

AI assisted in:

- Identifying potential edge cases and infinite loops in the cellular automata state transition rules during early testing.
- Coding and debugging of python files.
- Formatting this document and rewording initial drafts to adhere to conference-level expectations.

8 Conclusions

The synchronization bottleneck at partition boundaries fundamentally limits the scalability of spatial simulations. This project demonstrates that if the spatial environment can be mapped to a strict deterministic hierarchy, distributed nodes do not need to rely on conservative network blocking or memory-intensive Time Warp rollbacks. By employing an Asynchronous Rule-Based Coupling protocol, distributed systems can resolve spatial collisions in $\mathcal{O}(1)$ local compute time. Empirical benchmarking proves this architecture effectively isolates CPU throughput from network latency, allowing highly dense robotic simulations to execute nearly 100 times faster under heavy network lag than standard conservative alternatives.

9 Lessons Learned

In early phases, abstract state hierarchies (e.g., Solid > Liquid > Gas) were proposed, but implementing these revealed extreme vulnerability to cyclic infinite loops. Pivoting to mathematically rigorous, integer-based deterministic states (e.g., $\max()$ evaluation) proved that algorithms intended for distributed systems must prioritize absolute transitive consistency over abstract theoretical concepts. Observing the exponential degradation of the conservative model highlighted how theoretical assumptions of "fast local networks" completely break down in practice under minor spatial loads.

Furthermore, applying this protocol to two distinct simulation types revealed a critical limitation regarding temporal causality. In the Forest Fire model, the deterministic resolution instantly forces the highest energy state (e.g., $FIRE > TREE$) without regard for the exact simulation tick the fire actually ignited. This breaks the temporal timeline, causing the simulated fire to propagate faster or slower than the rules dictate purely due to network lag. This realization proved that Asynchronous Deterministic Coupling is poorly suited for continuous physical phenomena where strict timelines matter. However, it is exceptionally well-suited for heuristic pathfinding (like the Warehouse Robot model), where the ultimate goal is spatial convergence and collision avoidance rather than historical accuracy.

References

- [1] Andelfinger, P., and Uhrmacher, A. *Synchronous speculative simulation of tightly coupled agents in continuous time on CPUs and GPUs*. SIMULATION, 2023.
- [2] Carothers, C. D., Bauer, D., and Pearce, S. *ROSS: A High-Performance, Low-Memory, Modular Time Warp System*. Journal of Parallel and Distributed Computing, 2002.
- [3] Fujimoto, R. M. *Parallel Discrete Event Simulation*. ACM Computing Surveys, vol. 22, no. 1, pp. 1-45, 1990.
- [4] Huang, Z., et al. *Distributed Simulation for Digital Twins of Large-Scale Real-World DiffServ-Based Networks*. arXiv preprint arXiv:2405.20815, 2024.
- [5] Xia, W., et al. *Dynamic Load Balancing Based on Hypergraph Partitioning for Parallel Geospatial Cellular Automata Models*. ISPRS International Journal of Geo-Information, 2025.

Appendix: Software and Implementation Manual

This project was developed entirely in Python 3. The codebase is divided into several modules to cleanly separate the simulation logic from the distributed network architecture.

A. Software Developed

- **simulation.py:** The core simulation engine containing both the `ForestFireSim` and `WarehouseRobotSim` classes. It handles the 2D grid matrix, the $O(1)$ heuristic pathfinding, and the `resolve_boundary_conflict` deterministic logic.
- **worker.py:** The daemon script deployed to individual distributed nodes. It hosts the `WorkerNodeService` gRPC server, advances the local matrix, and handles the asynchronous background thread for transmitting boundary arrays to peers.
- **master.py:** The orchestration script acting as the global benchmark timer. It broadcasts `TickRequest` events via multi-threaded futures to ensure all workers are triggered simultaneously, and calculates final throughput metrics.
- **run_benchmarks.py:** A fully automated test harness. It utilizes the `subprocess` module to automatically bind Worker A and Worker B to separate system ports, iteratively loop through the different latency parameters and simulation engines, and securely terminate the cluster processes upon completion.
- **protocol.proto:** The schema definition file dictating the `TickRequest` and `BoundaryData` data structures.

B. Open Source Software Used

- **grpcio and grpcio-tools:** Provided the underlying HTTP/2 TCP transport layer and the `protoc` compiler used to generate the Python stubs (`protocol_pb2.py` and `protocol_pb2_grpc.py`).
- **matplotlib and numpy:** Utilized exclusively in the `visualization.py` script to plot the generated terminal benchmark metrics into the high-resolution vector line charts presented in this report.
- **concurrent.futures:** A standard Python library utilized heavily in both the master and worker nodes to circumvent the Global Interpreter Lock (GIL) and allow true asynchronous network broadcasting.

C. Core Protocol Implementations

To strictly adhere to the project requirements regarding original code attribution, the following snippets represent the entirely original core architecture developed for this research.

1. The gRPC Protobuf Schema (`protocol.proto`)

This schema dictates the highly compressed binary payloads exchanged between the distributed nodes.

```
syntax = "proto3";  
package distributed_sim;
```

```

enum CellState {
    EMPTY = 0;
    TREE = 1;
    FIRE = 2;
}

service WorkerNode {
    rpc ComputeTick (TickRequest) returns (TickResponse);
    rpc ReceiveBoundary (BoundaryData) returns (BoundaryAck);
}

message BoundaryData {
    int32 tick_number = 1;
    string sender_id = 2;
    repeated CellState edge_cells = 3;
}

```

2. Deterministic Conflict Resolution Engine

The following Python implementation demonstrates the $O(1)$ memoryless state convergence applied by the worker nodes during a boundary conflict.

```

def resolve_boundary_conflict(self, peer_boundary: list, is_left: bool) -> bool:
    """
    Applies the static deterministic hierarchy (e.g., FIRE > TREE > EMPTY)
    to the boundary edge in  $O(1)$  time without rolling back memory.
    """
    col = self.width - 1 if is_left else 0
    conflict = False

    for r in range(self.height):
        local_state = self.grid[r][col]
        peer_state = peer_boundary[r]

        # If network asynchrony caused divergence
        if local_state != peer_state:
            conflict = True
            # The mathematical resolution tie-breaker
            self.grid[r][col] = max(local_state, peer_state)

    return conflict

```